Practical – 3

Study and implement PCA in python.

1. What is Principle Component Analysis (PCA) ?

PCA (Principal Component Analysis) is a dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information. It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components.

2. How PCA works?

PCA uses linear algebra to transform data into new features called principal components. It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix. PCA selects the top components with the highest eigenvalues and projects the data onto them simplify the dataset.

Note: It prioritizes the directions where the data varies the most because more variation = more useful information.

Imagine you're looking at a messy cloud of data points like stars in the sky and want to simplify it. PCA helps you find the "most important angles" to view this cloud so you don't miss the big patterns.

3. Implementation of PCA using Eigenvalues and Eigenvectors:

Step 1: Dataset Preparation

We are using wine data for this practical it contains 13 features and 178 rows

Step 1: Load the wine dataset from sklearn library

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_wine
wine = load_wine()
X = wine.data
y = wine.target
feature_names = wine.feature_names
print("Initial Shape of our Dataset", X.shape)
```

```
Initial Shape of our Dataset (178, 13)
```

Step 2: Perform Standardization on the dataset:-

We need to perform standardization as the PCA works on variance and if the data is not standardized then it can create a greate problem to our Analysis.

Standardization of the data

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 3: Compute the Covariance Matrix

Calculate the covariance matrix to understand relationships between features. We do transpose of our dataset as the numpy expects features in rows.

Covariance Matrix

```
cov_matrix = np.cov(X_scaled.T)
print("Covariance matrix shape:", cov_matrix.shape)
```

```
Covariance matrix shape: (13, 13)
```

Step 4: Compute Eigenvalues and Eigenvectors

Find eigenvalues (variance amount) and eigenvectors (principal directions) of the covariance matrix.

Eigenvalues and Eigenvectors

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

print("Eigenvalues:\n", eigenvalues)
print("\nEigenvectors shape:", eigenvectors.shape)
```

```
Eigenvalues:
 [4.73243698 2.51108093 1.45424187 0.92416587 0.85804868 0.64528221
 0.55414147 0.10396199 0.35046627 0.16972374 0.29051203 0.22706428
 0.25232001]

Eigenvectors shape: (13, 13)
```

Step 5: Sort Eigenvalues and Eigenvectors:-

Arrange them in descending order of eigenvalues to rank principal components by importance

Sort Eigenvalues and Eigenvectors

```
sorted_index = np.argsort(eigenvalues)[::-1]
sorted_eigenvalues = eigenvalues[sorted_index]
sorted_eigenvectors = eigenvectors[:, sorted_index]
```

Step 6: Calculate Explained Variance Ratio:-

Determine how much variance each principal component explains.

Calculate Explained Variance Ratio

```
explained_variance_ratio = sorted_eigenvalues / np.sum(sorted_eigenvalues)

print("\nExplained Variance Ratio:")
for i, var in enumerate(explained_variance_ratio):
    print(f"PC{i+1}: {var:.4f}")
```

```
Explained Variance Ratio:
PC1: 0.3620
PC2: 0.1921
PC3: 0.1112
PC4: 0.0707
PC5: 0.0656
PC6: 0.0494
PC7: 0.0424
PC8: 0.0268
PC9: 0.0222
PC10: 0.0193
PC11: 0.0174
PC12: 0.0130
PC13: 0.0080
```

Step 7: Select Top k Principal Components

Choose the top eigenvectors corresponding to the largest eigenvalues for dimensionality reduction.

Select Top k Components

```
k = 7
top_k_eigenvectors = sorted_eigenvectors[:, :k]
```

Step 8: Transform the Data (Projection) and Print Dimensions:

Project the standardized data onto the selected principal components using dot product. Print the dimensions of the reduced data.

Transform the Data (Projection)

```
X_pca = X_scaled.dot(top_k_eigenvectors)

print("\nShape after PCA transformation:", X_pca.shape)
print("First 5 rows of transformed data:\n", X_pca[:5])
```

```
Shape after PCA transformation: (178, 7)
First 5 rows of transformed data:
 [[-3.31675081  1.44346263 -0.16573904  0.21563119  0.69304284  0.22388013
  -0.59642655]
 [-2.20946492 -0.33339289 -2.02645737  0.29135832 -0.25765463  0.92712024
  -0.05377561]
 [-2.51674015  1.0311513   0.98281867 -0.72490231 -0.25103312 -0.54927605
  -0.42420545]
 [-3.75706561  2.75637191 -0.17619184 -0.56798331 -0.31184159 -0.114431
   0.3833373 ]
 [-1.00890849  0.86983082  2.02668822  0.40976579  0.2984575   0.4065196
  -0.44407446]]
```